

Faculty of Computers and Information Systems

Information Retrieval

Practical Laboratory

Week 10: Integrated Mini Search Engine

Integrate the previous labs into a small command-line search engine. Load real text files, rank a user query, display results, and evaluate queries that have relevance judgments.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Organize the retrieval pipeline into small Python modules.
- Load a local document collection and build an inverted index.
- Calculate TF-IDF and rank user queries with cosine similarity.
- Display document IDs, scores, and text snippets.
- Evaluate judged queries and document limitations.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

The pipeline is: text files → document loading → cleaning → tokenization → stop-word removal → vocabulary → inverted index → TF-IDF → query processing → cosine similarity → ranking → evaluation. Each stage has an explicit input and output that can be inspected.

`file_utils.py` loads only the docs folder and reports missing or empty collections. `preprocessing.py` replaces non-ASCII-letter runs with spaces, then lowercases, tokenizes, and removes the same stop words. For this English teaching collection, it produces the same retained tokens as Week 3.

`indexing.py` builds unique sorted postings. `tfidf.py` constructs the vocabulary, TF/DF/IDF, and weighted vectors. `ranking.py` reuses collection IDF for queries and computes cosine scores. The index is built and can be inspected; the simple dense ranker compares the query with all document vectors. Candidate pruning is an optional extension.

`main.py` loads and indexes once per session. Interactive searches reuse that model. Only positive-score results are displayed, with at most five documents. Evaluation at $k=3$ runs when a normalized exact query string has judgments. Unjudged queries still rank, but the program makes no relevance-quality claim for them.

The application uses ASCII word tokens, a fixed stop-word set, raw TF, unsmoothed natural-log IDF, and deterministic ID tie order. It does not implement stemming, phrase search, semantic matching, or a web interface. Rebuild the model after changing the collection, and review the judgments if document contents change.

Course material: Integration of the loaders, preprocessing, indexing, weighting, ranking, and evaluation developed in Weeks 1-9.

3. Python Concepts Used

- **Module imports:** Reuse functions with clear responsibilities and avoid running a demo during import.
- **Function parameters:** Pass the model, documents, and judgments to query handling.
- **File checks and exceptions:** Report missing input collections and reject judgments with unknown document IDs.

- **Interactive loops:** Accept repeated queries and exit with quit, EOF, or Ctrl+C.
- **String normalization and slicing:** Match judgment query keys and print short document snippets.

4. Worked Example

Run this complete integration example from inside the Week 10 folder. It uses the included modules and computes its result from docs, rather than hard-coding a ranking.

```
import os
from file_utils import load_documents
from tfidf import build_tfidf
from ranking import rank_query
documents = load_documents(os.path.join(os.getcwd(), "docs"))
vocabulary, counts, df, idf, vectors = build_tfidf(documents)
results = rank_query("python data analysis", vocabulary, idf, vectors)
for name, score in results[:3]:
    print(name, f"{score:.4f}")
```

The top IDs and scores agree with Week 8 because the collection, preprocessing, and formulas agree. The full main.py adds script-relative paths, a persistent interactive session, snippets, and optional evaluation. The complete program section includes all required module listings.

5. Step-by-Step Practical Task

Step 1: Inspect the ten docs files and three judged queries. Sketch the pipeline and assign each stage to a module.

Step 2: Trace file_utils and preprocessing; compare their retained tokens with the Week 3 function on the sample collection.

Step 3: Build the inverted index and model once. Verify that posting lengths equal the DF counts.

Step 4: Run python main.py for the default demonstration, then python main.py --interactive for repeated searches.

Step 5: Search the three judged queries and one new query. Explain why evaluation appears only for judged queries.

Step 6: Complete the starter integration using your earlier functions. Test a missing folder, an empty query, an unknown-term query, and a modified collection.

Execution: open a terminal in this week folder and run python main.py. On Windows, py -3 can replace python. All required files are included; no internet or third-party Python packages are needed.

For repeated queries, run python main.py --interactive. Type quit to exit.

For one custom query, run python main.py "python data analysis".

Use the supplied module files as the starting point during the two-hour integration session. The complete listings below are a reference; you do not need to retype every earlier function.

6. Complete Python Program

The listing below matches main.py exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

main.py

```
"""Run a demo, pass a query, or use --interactive for repeated search."""
import json
import os
import sys
from file_utils import load_documents
```

```

from preprocessing import tokenize
from indexing import build_index
from tfidf import build_tfidf
from ranking import rank_query
from evaluation import report

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def show_results(query, model, documents, judgments):
    vocabulary, counts, df, idf, vectors = model
    results = rank_query(query, vocabulary, idf, vectors)
    ranked = [name for name, score in results if score > 0]
    print("\nQuery:", query)
    print("Query tokens:", tokenize(query))
    if not ranked:
        print("No matching documents with a positive score.")
    for position, (name, score) in enumerate(results[:5], 1):
        if score > 0:
            print(f"{position}. {name} {score:.4f}")
            print(" ", documents[name].strip()[:90])
    # Judgments refer to exact query strings, ignoring case/whitespace.
    key = " ".join(query.lower().split())
    for case in judgments:
        if key == " ".join(case["query"].lower().split()):
            report("Evaluation", ranked, case["relevant"], k=3)
            return
    print("No predefined relevance judgments for this query.")

def main():
    try:
        documents = load_documents(os.path.join(BASE_DIR, "docs"))
    except (OSError, ValueError) as error:
        print("Collection error:", error)
        return
    index = build_index(documents)
    model = build_tfidf(documents)
    judgments_path = os.path.join(BASE_DIR, "data", "relevance.json")
    judgments = []
    if os.path.isfile(judgments_path):
        with open(judgments_path, "r", encoding="utf-8") as file:
            judgments = json.load(file)
        for case in judgments:
            unknown = set(case["relevant"]) - set(documents)
            if unknown:
                print("Judgments reference missing documents:", sorted(unknown))
                return
    print("Documents:", len(documents), "Vocabulary:", len(index))
    if "--interactive" in sys.argv:
        print("Enter a free-text query; quit exits.")
        while True:
            try:
                query = input("Search > ").strip()
            except (EOFError, KeyboardInterrupt):
                break
            if query.lower() == "quit":
                break
            if query:
                show_results(query, model, documents, judgments)
    else:
        query = " ".join(sys.argv[1:]) or "python data analysis"
        show_results(query, model, documents, judgments)

if __name__ == "__main__":
    main()

```

file_utils.py

```

"""Strict loader for the final project: no fallback collection."""
import os

def load_documents(folder_path):
    if not os.path.isdir(folder_path):
        raise ValueError("Create a docs folder containing UTF-8 .txt files")
    documents = {}
    for filename in sorted(os.listdir(folder_path)):
        path = os.path.join(folder_path, filename)
        if filename.endswith(".txt") and os.path.isfile(path):
            with open(path, "r", encoding="utf-8") as file:
                documents[filename[:-4]] = file.read()
    if not documents:
        raise ValueError("The docs folder contains no .txt documents")
    return documents

```

preprocessing.py

```

"""Retain the Week 3 tokens while making cleaning explicit."""
import re
STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

def clean_text(text):
    # Keep boundaries between words, including across punctuation.
    text = re.sub(r"^[a-zA-Z]+", " ", text)
    return re.sub(r"\s+", " ", text).strip().lower()

def tokenize(text):
    words = re.findall(r"[a-z]+", clean_text(text))
    return [word for word in words if word not in STOP_WORDS]

```

indexing.py

```

from preprocessing import tokenize

def build_index(documents):
    index = {}
    for name, text in documents.items():
        for word in tokenize(text):
            if word not in index:
                index[word] = set()
            index[word].add(name) # A document occurs at most once.
    for word in index:
        index[word] = sorted(index[word])
    return index

```

tfidf.py

```

import math
from preprocessing import tokenize

def term_frequency(tokens):
    counts = {}
    for term in tokens:
        counts[term] = counts.get(term, 0) + 1
    return counts

def build_tfidf(documents):
    """Raw TF and natural-log IDF; no smoothing or extra normalization."""
    counts = {}
    document_frequency = {}
    for name, text in documents.items():

```

```

        counts[name] = term_frequency(tokenize(text))
    for term in counts[name]:
        document_frequency[term] = document_frequency.get(term, 0) + 1
    vocabulary = sorted(document_frequency)
    idf = {}
    total = len(documents)
    for term in vocabulary:
        idf[term] = math.log(total / document_frequency[term])
    vectors = {}
    for name in sorted(documents):
        vectors[name] = [
            counts[name].get(term, 0) * idf[term] for term in vocabulary
        ]
    return vocabulary, counts, document_frequency, idf, vectors

```

ranking.py

```

import math
from preprocessing import tokenize
from tfidf import term_frequency

def dot_product(v1, v2):
    if len(v1) != len(v2):
        raise ValueError("Vectors must have the same length")
    return sum(v1[i] * v2[i] for i in range(len(v1)))

def magnitude(vector):
    return math.sqrt(sum(value * value for value in vector))

def cosine_similarity(v1, v2):
    product = dot_product(v1, v2)
    denominator = magnitude(v1) * magnitude(v2)
    if denominator == 0:
        return 0.0
    return product / denominator

def query_vector(query, vocabulary, idf):
    counts = term_frequency(tokenize(query))
    return [counts.get(term, 0) * idf[term] for term in vocabulary]

def rank_query(query, vocabulary, idf, vectors):
    vector = query_vector(query, vocabulary, idf)
    results = []
    for name, document_vector in vectors.items():
        score = cosine_similarity(vector, document_vector)
        results.append((name, score))
    # Stable, reproducible order for equal scores.
    results.sort(key=lambda item: (-item[1], item[0]))
    return results

```

evaluation.py

```

def precision(retrieved, relevant):
    retrieved = set(retrieved)
    relevant = set(relevant)
    if not retrieved:
        return 0.0
    return len(retrieved & relevant) / len(retrieved)

def recall(retrieved, relevant):
    relevant = set(relevant)
    if not relevant:
        return 0.0
    return len(set(retrieved) & relevant) / len(relevant)

def f1_score(p, r):
    if p + r == 0:

```

```

        return 0.0
    return 2 * p * r / (p + r)

def unique_ranking(ranked_docs):
    result = []
    for name in ranked_docs:
        if name not in result:
            result.append(name)
    return result

def precision_at_k(ranked_docs, relevant_docs, k):
    if k <= 0:
        raise ValueError("k must be positive")
    top = unique_ranking(ranked_docs)[:k]
    # Unfilled positions count as nonrelevant: denominator stays k.
    return len(set(top) & set(relevant_docs)) / k

def recall_at_k(ranked_docs, relevant_docs, k):
    if k <= 0:
        raise ValueError("k must be positive")
    return recall(unique_ranking(ranked_docs)[:k], relevant_docs)

def report(label, ranked_docs, relevant, k=3):
    top = unique_ranking(ranked_docs)[:k]
    true_positive = len(set(top) & set(relevant))
    false_positive = len(set(top) - set(relevant))
    false_negative = len(set(relevant) - set(top))
    p = precision(top, relevant)
    r = recall(top, relevant)
    print(label, "top:", top)
    print("TP, FP, FN:", true_positive, false_positive, false_negative)
    print(f"Precision={p:.4f} Recall={r:.4f} F1={f1_score(p, r):.4f}")
    print(f"P@{k}={precision_at_k(ranked_docs, relevant, k):.4f}")
    print(f"R@{k}={recall_at_k(ranked_docs, relevant, k):.4f}")

```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

Documents: 10 Vocabulary: 60

Query: python data analysis

Query tokens: ['python', 'data', 'analysis']

1. doc4 0.5123

Python supports data analysis with libraries like Pandas

2. doc9 0.4326

Data analysis explores data and summarizes patterns. Data science combines analysis and ma

3. doc1 0.3466

Python is great for data science and machine learning

4. doc6 0.0489

Database systems organize structured data in tables. Database queries retrieve data from t

5. doc7 0.0284

Computer networks connect devices using protocols. Networks transfer data between computer

Evaluation top: ['doc4', 'doc9', 'doc1']

TP, FP, FN: 3 0 0

Precision=1.0000 Recall=1.0000 F1=1.0000

P@3=1.0000

R@3=1.0000

These results use all ten shipped documents. Editing the collection changes the index and may change weights, ranks, and evaluation results.

8. Laboratory Exercises

1. Basic: change the display limit from five results to three without changing the metric cutoff.
2. Implementation: finish `search(query, documents)` in the starter using your earlier preprocessing, weighting, and cosine functions.
3. Debugging: deliberately change only query preprocessing. Use a test query to show why document/query consistency matters.
4. Interpretation: inspect the posting list and DF for data and explain how they support different stages of retrieval.
5. Implementation: add a new original document, rebuild the model, and explain why IDF and scores can change for existing documents.
6. Evaluation: define a new information need and relevance set before looking at the ranking, then calculate its metrics.
7. Extension: add the Week 6 Boolean query mode as a separate command and compare its set of matches with ranked retrieval.

9. Student Tasks

- A complete runnable project folder containing all modules, docs, relevance data, and a README with exact commands.
- A short report explaining the pipeline and the design choices, including preprocessing, weighting, and zero-result behavior.
- Computed rankings and evaluation results for all three supplied judged queries, plus one original judged query.
- A test transcript covering normal search, an unknown query, missing/empty collection handling, and consistent DF/posting counts.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. Which parts of the model can be reused for several queries?
2. Why must the model be rebuilt after documents change?
3. Why can an unjudged query be ranked but not evaluated against a known relevance set?
4. How do posting lists and weighted vectors serve different purposes?
5. Which limitations would you address first for a larger or multilingual collection?

11. Challenge Exercise

Optional: Use the union of posting lists for nonzero-weight query terms to select ranking candidates. Confirm that positive-score results match the all-document baseline, then compare the number of cosine calculations.